

# **Module 2**

## cocotb Fundamentals

# Learning objectives

- Master cocotb for hardware verification

# Prerequisites

- See module README

# Learning path

## Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["cocotb Architecture and Conc"]

T2["Simulator Integration"]

T1 --> T2

T3["Clock Generation and Managem"]

T2 --> T3

Master cocotb for hardware verification

# Overview

- This module provides comprehensive coverage of cocotb, the coroutine-based testbench framework that...

# How to learn this module

# Suggested learning path

- Finish Module 1 cocotb labs so clock/reset coroutine patterns are familiar
- Read ``module2/dut/registers/simple_register.v`` and ``shift_register.v`` port lists before writing stimuli
- Work through ``module2/examples/triggers/`` — ``RisingEdge``, ``FallingEdge``, and ``Timer`` are core to all
- Study ``module2/examples/clock_generation/`` and ``reset_patterns/`` — shared fixtures decouple timing
- No pyuvvm yet; tests are flat `@cocotb.test`` functions selected by Makefile `TEST=``

# **Design architecture**



# 1. RTL portfolio architecture

- `simple\_register.v` — 8-bit register, enable, synchronous reset (primary lab DUT)
- `shift\_register.v` — serial in/out, parallel tap; exercises multi-cycle behavior
- `simple\_fifo.v` — 16×8 FIFO with full/empty flags (reference for extension)
- `simple\_fsm.v` — IDLE/START/WORK/DONE four-state controller (optional study)

## Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
  subgraph regs["Registers"]
    SR[simple_register]
    SH[shift_register]
  end
  end
  subgraph other["Reference DUTs"]
```

## 2. cocotb testbench architecture

- Test module → coroutines  
    (``@cocotb.test``) → DUT signals  
    via ``dut.signal`` handles
- Clock generators in examples  
    decouple timing from test logic
- No UVM hierarchy; flat Python  
    functions and shared fixtures  
    via Makefile ``TEST=`` selection
- Concurrent coroutines use  
    triggers to model parallel  
    clock, reset, and stimulus  
    threads

### Verification Architecture

Diagram: Verification Architecture  
(render with mmdc when available)  
flowchart LR  
TB[cocotb @test coroutines] --> DRV[drive dut ports]  
DRV --> DUT[(register / shift DUT)]  
DUT --> MON[sample on triggers]  
MON --> CHK[assert + log]

### 3. Execution pipeline

- Build: Verilator compiles selected DUT under  
`module2/dut/registers/` per Makefile top module
- Sim: cocotb schedules coroutines on triggers; reset  
coroutines establish known state before stimulus
- Check: Python assertions on `dut.q.value` / shift  
outputs; cocotb regression aggregates `results.xml...
- Examples path: `module2.sh --triggers` runs tutorial  
makes without full DUT regression

## 4. Example vs test separation

- ``module2/examples/`` — signal access, clocks, triggers, reset patterns (runnable tutorials)
- ``module2/tests/cocotb_tests/`` — regression tests bound to specific DUTs
- Orchestrator ``scripts/module2.sh`` routes flags to examples or cocotb makes
- Reference FIFO/FSM RTL supports self-study; graded labs focus on register and shift register

# DUT — simple register

```
/**
 * Module 2: Simple Register
 *
 * A basic register with clock, reset, and enable.
 *
 * Ports:
 *   clk:      Clock signal
 *   rst_n:    Active-low reset
 *   enable:   Register enable
 *   d:        Data input
 *   q:        Data output
 */

module simple_register (
    input  wire      clk,
    input  wire      rst_n,
    # ...
```

# cocotb — reset then write

```
"""
Module 2 Test: Simple Register
Comprehensive testbench for simple_register module.
"""

import cocotb
from cocotb.clock import Clock
from cocotb.triggers import Timer, RisingEdge, FallingEdge
from cocotb.binary import BinaryValue

async def reset_dut(dut, duration_ns=50, propagation_delay_ns=10):
    """
    Reset the DUT.

    Args:
        # ...
```

# **Key files to study**

# Open these in the repo

- ``module2/dut/registers/simple_register.v`` — 8-bit register with enable and synchronous reset (prima...
- ``module2/dut/registers/shift_register.v`` — serial in/out with parallel tap; multi-cycle behavior
- ``module2/examples/triggers/triggers_example.py`` — cocotb trigger primitives and concurrency
- ``module2/tests/cocotb_tests/test_simple_register.py`` — directed write/read and boundary values
- ``module2/tests/cocotb_tests/test_shift_register.py`` — serial shift timing regression
- ``scripts/module2.sh`` — routes example makes and ``--cocotb-tests`` regressions



# **Verification & testing methods**

# 1. Stimulus and observation

- Directed writes/reads on register ports; shift tests exercise serial timing
- ` RisingEdge` / ` FallingEdge` / ` Time r` triggers structure concurrent activity
- Reset tests run first in regression order to establish known state

## Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

R[Reset test first] --> W[Directed writes]

W --> E[Edge / boundary values]

E --> REG[cocotb regression]

REG --> OK{All pass?}

## 2. Regression and Makefile flow

- ``make SIM=verilator TEST=test_simple_register``  
selects cocotb test module
- Regression runner executes multiple tests; pass/fail  
per ``cocotb.regression``
- Boundary tests (``test_register_all_values``) cover  
0x00, 0x7F, 0x80, 0xFF

### 3. Step-by-step lab execution

- 1. Tutorials: ``./scripts/module2.sh --signal-access --triggers --reset-patterns``
- 2. Register lab: ``cd module2/tests/cocotb_tests && make SIM=verilator TEST=test_simple_register``
- 3. Shift register lab: ``make SIM=verilator TEST=test_shift_register`` — verify serial-to-parallel ti...
- 4. Full DUT regression: ``./scripts/module2.sh --cocotb-tests``
- 5. Debug failures with waves/logging before MODULE2 assessment

## 4. Debug and closure

- Logging via ``cocotb.log``; VCD from simulator flags for waveform debug
- ``./scripts/module2.sh --cocotb-tests`` validates key cocotb tests and example makes
- Assessment: clocks, triggers, reset sequences, structured cocotb tests

# Regression Makefile

```
# Makefile for Module 2 cocotb tests
# Usage: make SIM=verilator test_simple_register

# Default simulator
SIM ?= verilator

# Python files
PYTHON_FILES = test_simple_register.py test_shift_register.py

# Verilog files
VERILOG_FILES = ../../dut/registers/simple_register.v
               ../../dut/registers/shift_register.v

# Test to run
TEST ?= test_simple_register
```

# Cocotb variables

# Syllabus topics

# 1. cocotb Architecture and Concepts (1/3)

- What is cocotb?
- Coroutine-based testbench framework
- Python testbenches for Verilog/VHDL
- Simulator abstraction
- History and motivation
- cocotb Architecture



# 1. cocotb Architecture and Concepts (2/3)

- Python testbench layer
- Simulator interface layer
- DUT interaction
- Event scheduling
- Key Concepts
- Coroutines for simulation

# 1. cocotb Architecture and Concepts (3/3)

- Triggers for synchronization
- Handles for signal access
- Simulation time management

## 2. Simulator Integration (1/3)

- Supported Simulators
- Verilator (recommended)
- Icarus Verilog
- ModelSim/QuartaSim
- GHDL (VHDL)
- VCS, Xcelium (commercial)

## 2. Simulator Integration (2/3)

- Simulator Selection
- Environment variables
- Makefile configuration
- Simulator-specific features
- Compilation Process
- Verilog compilation

## 2. Simulator Integration (3/3)

- cocotb library compilation
- Linking process
- Makefile structure

# 3. Clock Generation and Management (1/3)

- Clock Generation
- `Clock` class usage
- Clock parameters (period, units)
- Starting clocks
- Multiple clocks
- Clock Patterns

# 3. Clock Generation and Management (2/3)

- Regular clocks
- Gated clocks
- Clock division
- Clock stopping
- Clock Domain Management
- Multiple clock domains

# 3. Clock Generation and Management (3/3)

- Clock domain crossing
- Synchronization between domains



## 4. Signal Access and Driving (1/4)

- Signal Handles
- Accessing DUT signals
- ``dut.signal_name`` syntax
- Signal value types
- Signal properties
- Reading Signal Values

## 4. Signal Access and Driving (2/4)

- ``.value`` property
- Integer conversion
- Binary representation
- Signal state checking
- Driving Signals
- Assigning values

## 4. Signal Access and Driving (3/4)

- Integer assignment
- Binary string assignment
- High-impedance (Z) and unknown (X)
- Signal Types
- Single-bit signals
- Multi-bit signals (vectors)

## 4. Signal Access and Driving (4/4)

- Buses and arrays
- Bidirectional signals

## 5. Triggers and Coroutines (1/4)

- Trigger Types
- `` RisingEdge(signal)``
- `` FallingEdge(signal)``
- `` Edge(signal)`` (any edge)
- `` Timer(time, units)``
- `` ReadOnly()`` (end of time step)

## 5. Triggers and Coroutines (2/4)

- `ReadWrite()` (during time step)
- `Combine(\*triggers)` (multiple triggers)
- `First(\*triggers)` (first to occur)
- Coroutine Execution
- Defining coroutines
- Starting coroutines

## 5. Triggers and Coroutines (3/4)

- ``cocotb.start_soon()`` vs ``await``
- Parallel execution
- Coroutine Synchronization
- Waiting for triggers
- Coordinating multiple coroutines
- Timeout handling

## 5. Triggers and Coroutines (4/4)

- Exception propagation



## 6. Test Structure and Organization (1/3)

- Test Function Structure
- `@cocotb.test()` decorator
- Test function signature
- DUT parameter
- Test organization
- Test Phases

## 6. Test Structure and Organization (2/3)

- Setup phase
- Test execution
- Cleanup phase
- Error handling
- Multiple Tests
- Test discovery

## 6. Test Structure and Organization (3/3)

- Test selection
- Test parameters
- Test fixtures

# 7. Reset and Initialization (1/2)

- Reset Strategies
  - Synchronous reset
  - Asynchronous reset
- Reset sequences
- Reset verification
- Initialization Patterns

## 7. Reset and Initialization (2/2)

- Signal initialization
- State initialization
- Configuration setup
- Initial conditions

## 8. Common Verification Patterns (1/3)

- Stimulus Generation
- Sequential patterns
- Random patterns
- Constrained random
- File-based stimulus
- Response Checking

## 8. Common Verification Patterns (2/3)

- Immediate checking
- Deferred checking
- Reference model comparison
- Scoreboard patterns
- Transaction-Level Modeling
- Transaction classes

## 8. Common Verification Patterns (3/3)

- Transaction generation
- Transaction execution
- Transaction checking



## 9. Debugging with cocotb (1/4)

- Logging and Reporting
  - cocotb logging
  - Log levels
  - Log formatting
  - Debug messages
- Waveform Generation

## 9. Debugging with cocotb (2/4)

- VCD file generation
- FST file generation
- Waveform viewing
- Signal tracing
- Interactive Debugging
- Python debugger (pdb)

## 9. Debugging with cocotb (3/4)

- Breakpoints
- Variable inspection
- Step-through debugging
- Common Issues
- Signal access errors
- Timing issues

## 9. Debugging with cocotb (4/4)

- Simulation hangs
- Value conversion problems

# 10. Advanced cocotb Features (1/3)

- Memory Access
- Memory modeling
- Memory initialization
- Memory access patterns
- Bus Functional Models (BFM)
- BFM concepts

## 10. Advanced cocotb Features (2/3)

- BFM implementation
- Reusable BFMs
- Performance Optimization
- Coroutine efficiency
- Trigger optimization
- Simulation speed

# 10. Advanced cocotb Features (3/3)

- Memory usage

# 11. Integration with pytest (1/2)

- pytest Integration
- Using pytest with cocotb
- Test discovery
- Fixtures
- Parametrization
- Test Organization



# 11. Integration with pytest (2/2)

- Test directory structure
- Test naming conventions
- Test grouping
- Test execution

# **Command reference highlights**

# cocotb examples (tutorial makes)

- ``./scripts/module2.sh --triggers`` — run trigger tutorial under ``module2/examples/triggers/``
- ``./scripts/module2.sh --clock-generation --reset-patterns`` — timing and reset fixture drills
- ``./scripts/module2.sh`` — all five example tracks (signal access through common patterns)

# DUT regression tests

- ``./scripts/module2.sh --cocotb-tests`` — register and shift-register makes via orchestrator
- ``cd module2/tests/cocotb_tests && make SIM=verilator TEST=test_simple_register``
- ``make SIM=verilator TEST=test_shift_register`` — serial timing and parallel output checks

# Debug and waves

- ``make SIM=verilator TEST=test_simple_register`  
    `WAVES=1`` — enable VCD/FST when Makefile supports it
- Use ``cocotb.log`` verbosity and simulator stdout from  
    ``results.xml`` for pass/fail triage
- Re-run single ``TEST=`` target after RTL or stimulus  
    edits — faster than full example sweep

# Hands-on examples

# Module 2 orchestrator

```
$ ./scripts/module2.sh --help
```

Terminal: module2\_check

```
[[0;36m===== [[0m
[[0;36mModule 2: cocotb Fundamentals [[0m
[[0;36m===== [[0m
```

Usage: ./scripts/module2.sh [OPTIONS]

Module 2: cocotb Fundamentals

This script runs examples and tests for Module 2.

## OPTIONS:

### Examples:

--signal-access Run signal access examples  
--clock-generation Run clock generation examples  
--triggers Run trigger usage examples  
--reset-patterns Run reset pattern examples  
--common-patterns Run common verification pattern examples  
--all-examples Run all examples (default)  
--skip-examples Skip all examples

### Tests:

--cocotb-tests Run cocotb tests

### Environment:

--venv DIR Virtual environment directory (default: .venv)  
--no-venv Don't use virtual environment  
--sim SIMULATOR Simulator to use (default: verilator)

### Other:

--help, -h Show this help message

## EXAMPLES:

# Run all examples  
./scripts/module2.sh  
  
# Run specific examples

# Exercise scaffold

```
./scripts/module2.sh --scaffold
```



# Demo: Signal Access

```
$ .venv/bin/python  
    module2/examples/signal_access/signal_access_example.py
```

Terminal: ex01\_signal\_access

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module2/examples/signal\_a  
import cocotb

ModuleNotFoundError: No module named 'cocotb'

# Demo: Clock Generation

```
$ .venv/bin/python  
    module2/examples/clock_generation/clock_generation_example.py
```

Terminal: ex02\_clock\_generation

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module2/examples/clock\_ge  
import cocotb

ModuleNotFoundError: No module named 'cocotb'

# Demo: Triggers

```
$ .venv/bin/python module2/examples/triggers/triggers_example.py
```

Terminal: ex03\_triggers

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module2/examples/triggers

import cocotb

ModuleNotFoundError: No module named 'cocotb'

# Demo: Reset Patterns

```
$ .venv/bin/python  
    module2/examples/reset_patterns/reset_patterns_example.py
```

Terminal: ex04\_reset\_patterns

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module2/examples/reset\_pa  
import cocotb

ModuleNotFoundError: No module named 'cocotb'

# Demo: Common Verification Patterns

```
$ .venv/bin/python  
    module2/examples/common_patterns/common_patterns_example.py
```

Terminal: ex05\_common\_patterns

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module2/examples/common\_p  
import cocotb

ModuleNotFoundError: No module named 'cocotb'

# Practice & assessment

# What you should know (1/3)

- Understand cocotb architecture
- Integrate with simulators
- Generate and manage clocks
- Access and drive signals
- Use triggers effectively
- Structure tests properly

# What you should know (2/3)

- Implement reset sequences
- Use common verification patterns
- Debug cocotb testbenches
- Optimize testbench performance
- Signal handles
- Value reading



# What you should know (3/3)

- Signal types

# Exercises

- Clock Management
- Signal Operations
- Trigger Patterns
- Test Structure
- Debugging

# Assessment checklist

- Understands cocotb architecture
- Can integrate with simulators
- Can generate and manage clocks
- Can access and drive signals
- Can use triggers effectively
- Can structure tests properly

# Summary & next steps

- Master cocotb for hardware verification
- Complete module2/CHECKLIST.md
- Review module2/EXAMPLES.md and run each lab
- Next: Next module in course